

M02 - SQL Foundations

14-lesson learner book | Beginner-first professional institute standard | B01-style teaching contract

Purpose

Translate business questions into correct foundational SQL and validate the result.

Course boundary

This is the independent Standalone Data Engineer course, not Four-Course C3. Complete the course in sequence; each new idea is still taught from first principles.

Option B institute identity | Charcoal #1F2937 | Teal #14B8A6 | Soft Blue #3B82F6

This book is your teacher

Use one lesson at a time. Do not read protected review before saving a genuine attempt. Every lesson follows the same beginner-first sequence even when the engineering idea becomes more technical.

Daily lesson loop

1) why it matters -> 2) what you learn -> 3) prerequisite -> 4) picture it -> 5) new words -> 6) useful visual/source -> 7) follow once -> 8) expected result -> 9) why it worked -> 10) mistakes -> 11) recovery -> 12) independent task -> 13) validate -> 14) teach it back -> 15) protected review + changed retry -> 16) evidence/mastery + exact next route.

Prerequisite check

Complete M01 data foundations. You should already understand row grain, data types, keys, NULL/missing values, and basic validation thinking.

Attempt protection

A protected review is repair material, not a shortcut. Save the first attempt, open only the matching review if needed, record the repair, close review, then solve a changed retry from a clean state.

Contents / Index

Every formal lesson and its actual page number in this exact rebuilt book.

DE02-01 - Database, table, row, column and query mental model	5
DE02-02 - SELECT, aliases and LIMIT	7
DE02-03 - WHERE and comparison operators	9
DE02-04 - AND, OR, NOT and parentheses	11
DE02-05 - ORDER BY, DISTINCT and result inspection	13
DE02-06 - NULL, IS NULL and COALESCE	15
DE02-07 - Arithmetic expressions and safe calculations	17
DE02-08 - Aggregate functions	19
DE02-09 - GROUP BY and grouped grain	21
DE02-10 - HAVING after aggregation	23
DE02-11 - CASE for business categories	25
DE02-12 - String and date basics	27
DE02-13 - Simple subqueries and IN/EXISTS awareness	29
DE02-14 - Debugging and validation workflow	31

Prerequisite, tool and outcome map

ASSUMED

Complete M01 data foundations. You should already understand row grain, data types, keys, NULL/missing values, and basic validation thinking.

MODULE PURPOSE

Translate business questions into correct foundational SQL and validate the result.

EXIT EVIDENCE

Module practical + fresh retest + saved evidence; no critical unresolved failure.

Protected study route

Teaching -> assigned visual/source when useful -> follow once -> close worked material -> independent attempt -> validate -> save evidence -> teach it back. If weak: matching protected review -> targeted repair -> close review -> changed fresh retry -> revalidate.

DE02-01 - Database, table, row, column and query mental model

1. Why it matters

Install/verify MySQL 8+ and MySQL Workbench, load the course database, and explain the difference between a database, table, row, column, query and result grid. This matters because SQL can execute without error while filtering, grouping or calculating the wrong business population.

2. What you are learning

Install/verify MySQL 8+ and MySQL Workbench, load the course database, and explain the difference between a database, table, row, column, query and result grid.

3. Prerequisite / what you already know

Complete M01 data foundations. You should already understand row grain, data types, keys, NULL/missing values, and basic validation thinking.

4. Picture it in your head

Think of a database as a library, a table as one organized shelf, rows as individual records, columns as labeled attributes, and a query as a precise request for a particular result.

5. New words before we use them

A database stores structured objects. A table is a named set of rows and columns. In this lab, sales_orders has one row per order.

6. Useful video / source

Source: Alex The Analyst — SQL Beginner to Advanced

Assigned scope: 0:52–11:19

How to use it: Watch the assigned visual first, then use the Teaching Book for the course-specific build. Close the visual before the independent task.

OPEN VIDEO / SOURCE - CLICK HERE ([youtube.com](https://www.youtube.com/watch?v=OT1RERkfLNQ))

URL: <https://www.youtube.com/watch?v=OT1RERkfLNQ>

Do this after watching

Close the external source. Return to the bundled lesson and perform the exact Follow with me once steps from your own keyboard. Save evidence from your course data/tool, not a screenshot of the video.

7. Follow with me once

USE stage02_retail_lab;

SELECT DATABASE() AS current_database;

SELECT COUNT(*) AS total_rows FROM sales_orders;

Run the setup script. Confirm the active database is stage02_retail_lab and the table has exactly 30 rows. Open DESCRIBE sales_orders and point to order_id, qty, status and sales_rep.

8. Expected result / what you should see

Database = stage02_retail_lab; row count = 30; order_id is the primary key; sales_rep can be NULL.

9. Why it worked

The spreadsheet result is trustworthy because the source structure/formula logic is preserved and an independent cell/row/total check proves the displayed result came from the intended calculation. Your independent check is: Exactly 3 selected columns and 5 rows. The proof is the result grain and control query, not a plausible-looking result grid.

10. Common beginner mistakes

If the database/table is missing, rerun only the setup script and confirm no error occurred before writing learning queries. Do not install WSL for this module; M02 uses MySQL on Windows.

11. If you get stuck - recovery ladder

1. Return to the lesson prerequisite/starting state; do not stack fixes on an unknown state. 2. Rebuild the mental picture: Think of a database as a library, a table as one organized shelf, rows as individual records, columns as labeled attributes, and a query as a precise request for a particular result. 3. Run the earliest independent control: Exactly 3 selected columns and 5 rows. 4. Repair only the first failed step, rerun that same control, and save the failed attempt before opening protected Review.

12. Now you do a small version alone

With the video closed, open a new query tab and return order_id, region and product for any 5 rows. In a SQL comment, state the result grain.

13. Validate your result

Exactly 3 selected columns and 5 rows. Grain statement says one result row represents one order from sales_orders.

14. Teach it back in your own words

Without notes, explain Database, table, row, column and query mental model in your own words. Start with the purpose: Install/verify MySQL 8+ and MySQL Workbench, load the course database, and explain the difference between a database, table, row, column, query and result grid. Then name one realistic failure mode and explain how you would prove the result using this check: Exactly 3 selected columns and 5 rows.

15. Protected review + changed fresh retry

Save the genuine independent attempt first. If it is weak, open only the matching protected Review, identify the exact gap, record the repair, then close Review. Do a changed retry for Database, table, row, column and query mental model with a different input/identifier/condition and validate it independently. Work completed while Review is open is repair evidence, not mastery evidence.

16. Evidence / mastery + exact next route

Mastery evidence: Complete the matching guided/independent work with review material closed; save the actual result/error and validation evidence. Exact next route: DE02-02.

DE02-02 - SELECT, aliases and LIMIT

1. Why it matters

Choose output columns, calculate a simple expression, rename output with AS, and limit the number of returned rows. This matters because SQL can execute without error while filtering, grouping or calculating the wrong business population.

2. What you are learning

Choose output columns, calculate a simple expression, rename output with AS, and limit the number of returned rows.

3. Prerequisite / what you already know

Prerequisite: DE02-01 mastered with saved evidence.

4. Picture it in your head

SELECT is the projection window: choose exactly which columns/expressions should appear, give unclear expressions useful aliases, then inspect only a bounded sample before trusting the result.

5. New words before we use them

SELECT controls the projection: which columns or expressions appear in the result. AS gives an output label. LIMIT reduces the result rows returned after the query logic has run; it does not delete source rows.

6. Useful video / source

Source: Alex The Analyst — SQL Beginner to Advanced

Assigned scope: 11:19–21:47 + 47:41–51:10

How to use it: Watch the assigned visual first, then use the Teaching Book for the course-specific build. Close the visual before the independent task.

[OPEN VIDEO / SOURCE - CLICK HERE \(youtube.com\)](https://www.youtube.com/watch?v=OT1RErkfLNQ)

URL: <https://www.youtube.com/watch?v=OT1RErkfLNQ>

Do this after watching

Close the external source. Return to the bundled lesson and perform the exact Follow with me once steps from your own keyboard. Save evidence from your course data/tool, not a screenshot of the video.

7. Follow with me once

SELECT order_id, qty, unit_price, qty * unit_price AS gross_sales FROM sales_orders LIMIT 10;
Return order_id, qty, unit_price and gross_sales for 10 rows. Rename product as item_name in a second query.

8. Expected result / what you should see

O1001 gross_sales = 1700; O1002 = 1350. First query returns 10 rows and 4 columns.

9. Why it worked

The query is trustworthy because the business question, row grain and transformation stages are explicit, then an independent row/key/total check challenges a plausible-but-wrong result. Your independent check is: First query returns 30 rows / 3 columns. The proof is the result grain and control query, not a plausible-looking result grid.

10. Common beginner mistakes

If the alias is unavailable in the output, check AS spelling and whether the expression has a comma before it. If LIMIT is ignored, confirm it is at the end of the SELECT statement.

11. If you get stuck - recovery ladder

1. Return to the lesson prerequisite/starting state; do not stack fixes on an unknown state. 2. Rebuild the mental picture: SELECT is the projection window: choose exactly which columns/expressions should appear, give unclear expressions useful aliases, then inspect only a bounded sample before trusting the result. 3. Run the earliest independent control: First query returns 30 rows / 3 columns. 4. Repair only the first failed step, rerun that same control, and save the failed attempt before opening protected Review.

12. Now you do a small version alone

Return order_id, order_date and product for every row, then create a separate query returning 7 rows with product renamed item_name.

13. Validate your result

First query returns 30 rows / 3 columns. Second returns 7 rows and has a column named item_name.

14. Teach it back in your own words

Without notes, explain SELECT, aliases and LIMIT in your own words. Start with the purpose: Choose output columns, calculate a simple expression, rename output with AS, and limit the number of returned rows. Then name one realistic failure mode and explain how you would prove the result using this check: First query returns 30 rows / 3 columns.

15. Protected review + changed fresh retry

Save the genuine independent attempt first. If it is weak, open only the matching protected Review, identify the exact gap, record the repair, then close Review. Do a changed retry for SELECT, aliases and LIMIT with a different input/identifier/condition and validate it independently. Work completed while Review is open is repair evidence, not mastery evidence.

16. Evidence / mastery + exact next route

Mastery evidence: Complete the matching guided/independent work with review material closed; save the actual result/error and validation evidence. Exact next route: DE02-03.

DE02-03 - WHERE and comparison operators

1. Why it matters

Translate a single business condition into WHERE filters using =, <>, >, >=, <, <=, BETWEEN and IN. This matters because SQL can execute without error while filtering, grouping or calculating the wrong business population.

2. What you are learning

Translate a single business condition into WHERE filters using =, <>, >, >=, <, <=, BETWEEN and IN.

3. Prerequisite / what you already know

Prerequisite: DE02-02 mastered with saved evidence.

4. Picture it in your head

WHERE is a gate in front of the result set: each row is tested against the condition and only rows that evaluate TRUE pass through.

5. New words before we use them

WHERE decides which source rows are eligible before grouping. Every returned row must satisfy the condition. A filter should be readable as the business rule you were given.

6. Useful video / source

Source: Alex The Analyst — SQL Beginner to Advanced

Assigned scope: 21:47–33:54

How to use it: Watch the assigned visual first, then use the Teaching Book for the course-specific build. Close the visual before the independent task.

[OPEN VIDEO / SOURCE - CLICK HERE \(youtube.com\)](https://www.youtube.com/watch?v=OT1RErkfLNQ)

URL: <https://www.youtube.com/watch?v=OT1RErkfLNQ>

Do this after watching

Close the external source. Return to the bundled lesson and perform the exact Follow with me once steps from your own keyboard. Save evidence from your course data/tool, not a screenshot of the video.

7. Follow with me once

```
SELECT order_id, region, product, qty, status FROM sales_orders WHERE status = 'Completed'
AND qty >
= 2;
```

Run the Completed AND qty > = 2 filter and a separate COUNT(*) using the identical WHERE clause.

8. Expected result / what you should see

22 rows; no Pending or Cancelled rows survive.

9. Why it worked

The query is trustworthy because the business question, row grain and transformation stages are explicit, then an independent row/key/total check challenges a plausible-but-wrong result. Your independent check is: Date-range result = 6 rows. The proof is the result grain and control query, not a plausible-looking result grid.

10. Common beginner mistakes

If the count differs, read the English rule and test each predicate separately. Do not edit several clauses at once.

11. If you get stuck - recovery ladder

1. Return to the lesson prerequisite/starting state; do not stack fixes on an unknown state. 2. Rebuild the mental picture: WHERE is a gate in front of the result set: each row is tested against the condition and only rows that evaluate TRUE pass through. 3. Run the earliest independent control: Date-range result = 6 rows. 4. Repair only the first failed step, rerun that same control, and save the failed attempt before opening protected Review.

12. Now you do a small version alone

Return orders dated 2026-07-10 through 2026-07-12 inclusive, then return only West orders with unit_price >= 850.

13. Validate your result

Date-range result = 6 rows. West/high-price result contains only West and unit_price >= 850.

14. Teach it back in your own words

Without notes, explain WHERE and comparison operators in your own words. Start with the purpose: Translate a single business condition into WHERE filters using =, <>, >, >=, <, <=, BETWEEN and IN. Then name one realistic failure mode and explain how you would prove the result using this check: Date-range result = 6 rows.

15. Protected review + changed fresh retry

Save the genuine independent attempt first. If it is weak, open only the matching protected Review, identify the exact gap, record the repair, then close Review. Do a changed retry for WHERE and comparison operators with a different input/identifier/condition and validate it independently. Work completed while Review is open is repair evidence, not mastery evidence.

16. Evidence / mastery + exact next route

Mastery evidence: Complete the matching guided/independent work with review material closed; save the actual result/error and validation evidence. Exact next route: DE02-04.

DE02-04 - AND, OR, NOT and parentheses

1. Why it matters

Combine conditions safely and use parentheses so Boolean logic matches the business request. This matters because SQL can execute without error while filtering, grouping or calculating the wrong business population.

2. What you are learning

Combine conditions safely and use parentheses so Boolean logic matches the business request.

3. Prerequisite / what you already know

Prerequisite: DE02-03 mastered with saved evidence.

4. Picture it in your head

Boolean conditions are logic gates. AND requires both gates open, OR needs either one, NOT flips the test, and parentheses decide which gates are evaluated together.

5. New words before we use them

AND requires both conditions. OR allows either. NOT reverses a condition.

6. Useful video / source

Source: Alex The Analyst — SQL Beginner to Advanced

Assigned scope: 21:47–33:54

How to use it: Watch the assigned visual first, then use the Teaching Book for the course-specific build. Close the visual before the independent task.

[OPEN VIDEO / SOURCE - CLICK HERE \(youtube.com\)](https://www.youtube.com/watch?v=OT1RErkfLNQ)

URL: <https://www.youtube.com/watch?v=OT1RErkfLNQ>

Do this after watching

Close the external source. Return to the bundled lesson and perform the exact Follow with me once steps from your own keyboard. Save evidence from your course data/tool, not a screenshot of the video.

7. Follow with me once

```
SELECT order_id, region, status FROM sales_orders WHERE status='Completed'
```

```
AND (region='
```

```
East'
```

```
OR region='
```

```
North'
```

```
);
```

Return Completed orders from East or North with explicit parentheses. Count the rows separately.

8. Expected result / what you should see

13 rows. Regions are only East/North and every status is Completed.

9. Why it worked

This works because combine conditions safely and use parentheses so Boolean logic matches the business request is made explicit before the result is accepted. Your independent check is: Both independent queries return the same order_ids. The proof is the result grain and control query, not a plausible-looking result grid.

10. Common beginner mistakes

If a North Pending row appears, inspect the parentheses. Translate the WHERE clause back into English before rerunning.

11. If you get stuck - recovery ladder

1. Return to the lesson prerequisite/starting state; do not stack fixes on an unknown state. 2. Rebuild the mental picture: Boolean conditions are logic gates. 3. Run the earliest independent control: Both independent queries return the same order_ids. 4. Repair only the first failed step, rerun that same control, and save the failed attempt before opening protected Review.

12. Now you do a small version alone

Return non-Cancelled orders from East or North. Then write a logically equivalent version using NOT status = 'Cancelled'.

13. Validate your result

Both independent queries return the same order_ids. Save a set/count comparison.

14. Teach it back in your own words

Without notes, explain AND, OR, NOT and parentheses in your own words. Start with the purpose: Combine conditions safely and use parentheses so Boolean logic matches the business request. Then name one realistic failure mode and explain how you would prove the result using this check: Both independent queries return the same order_ids.

15. Protected review + changed fresh retry

Save the genuine independent attempt first. If it is weak, open only the matching protected Review, identify the exact gap, record the repair, then close Review. Do a changed retry for AND, OR, NOT and parentheses with a different input/identifier/condition and validate it independently. Work completed while Review is open is repair evidence, not mastery evidence.

16. Evidence / mastery + exact next route

Mastery evidence: Complete the matching guided/independent work with review material closed; save the actual result/error and validation evidence. Exact next route: DE02-05.

DE02-05 - ORDER BY, DISTINCT and result inspection

1. Why it matters

Remove repeated result values with DISTINCT, sort intentionally, and use top-N inspection without confusing sorting with filtering. This matters because SQL can execute without error while filtering, grouping or calculating the wrong business population.

2. What you are learning

Remove repeated result values with DISTINCT, sort intentionally, and use top-N inspection without confusing sorting with filtering.

3. Prerequisite / what you already know

Prerequisite: DE02-04 mastered with saved evidence.

4. Picture it in your head

ORDER BY lines records up for inspection; DISTINCT collapses exact duplicate result rows. DISTINCT can hide a join problem, so use it only when uniqueness is the actual requirement.

5. New words before we use them

DISTINCT removes duplicate combinations from the query result. ORDER BY changes sequence. LIMIT keeps only the first N rows after sorting.

6. Useful video / source

Source: Alex The Analyst — SQL Beginner to Advanced

Assigned scope: 11:19–21:47 + 47:41–51:10

How to use it: Watch the assigned visual first, then use the Teaching Book for the course-specific build. Close the visual before the independent task.

[OPEN VIDEO / SOURCE - CLICK HERE \(youtube.com\)](https://www.youtube.com/watch?v=OT1RErkfLNQ)

URL: <https://www.youtube.com/watch?v=OT1RErkfLNQ>

Do this after watching

Close the external source. Return to the bundled lesson and perform the exact Follow with me once steps from your own keyboard. Save evidence from your course data/tool, not a screenshot of the video.

7. Follow with me once

SELECT DISTINCT region FROM sales_orders ORDER BY region;

SELECT order_id, product, qty*unit_price AS gross_sales FROM sales_orders ORDER BY gross_sales DESC LIMIT 5;

List unique regions alphabetically. In a separate query return the five highest gross-sales orders.

8. Expected result / what you should see

Regions = East, North, South, West. Highest gross order = O1020 at 3600.

9. Why it worked

This works because remove repeated result values with DISTINCT, sort intentionally, and use top-N inspection without confusing sorting with filtering is made explicit before the result is accepted. Your independent check is: Exactly Card, Cash, UPI. The proof is the result grain and control query, not a plausible-looking result grid.

10. Common beginner mistakes

For a top-N question, sort DESC before LIMIT. If DISTINCT returns more values than expected, inspect all selected columns: DISTINCT applies to the whole selected combination.

11. If you get stuck - recovery ladder

1. Return to the lesson prerequisite/starting state; do not stack fixes on an unknown state. 2. Rebuild the mental picture: ORDER BY lines records up for inspection; DISTINCT collapses exact duplicate result rows. 3. Run the earliest independent control: Exactly Card, Cash, UPI. 4. Repair only the first failed step, rerun that same control, and save the failed attempt before opening protected Review.

12. Now you do a small version alone

Return distinct payment methods alphabetically. Then return the three largest-quantity orders with a deterministic tie-break on order_id.

13. Validate your result

Exactly Card, Cash, UPI. Top qty begins O1021=10, O1026=9, O1007=8.

14. Teach it back in your own words

Without notes, explain ORDER BY, DISTINCT and result inspection in your own words. Start with the purpose: Remove repeated result values with DISTINCT, sort intentionally, and use top-N inspection without confusing sorting with filtering. Then name one realistic failure mode and explain how you would prove the result using this check: Exactly Card, Cash, UPI.

15. Protected review + changed fresh retry

Save the genuine independent attempt first. If it is weak, open only the matching protected Review, identify the exact gap, record the repair, then close Review. Do a changed retry for ORDER BY, DISTINCT and result inspection with a different input/identifier/condition and validate it independently. Work completed while Review is open is repair evidence, not mastery evidence.

16. Evidence / mastery + exact next route

Mastery evidence: Complete the matching guided/independent work with review material closed; save the actual result/error and validation evidence. Exact next route: DE02-06.

DE02-06 - NULL, IS NULL and COALESCE

1. Why it matters

Find missing values correctly, explain COUNT(*) versus COUNT(column), and use COALESCE only when a display/default rule is justified. This matters because SQL can execute without error while filtering, grouping or calculating the wrong business population.

2. What you are learning

Find missing values correctly, explain COUNT(*) versus COUNT(column), and use COALESCE only when a display/default rule is justified.

3. Prerequisite / what you already know

Prerequisite: DE02-05 mastered with saved evidence.

4. Picture it in your head

NULL is an empty “unknown/not supplied” slot, not zero or an empty string. IS NULL tests that state; COALESCE chooses the first available fallback value deliberately.

5. New words before we use them

NULL means no value is present; it is not zero and not empty text. Use IS NULL / IS NOT NULL rather than = NULL. COUNT(*) counts rows; COUNT(column) counts non-NULL values in that column.

6. Useful video / source

Source: Luke Barousse — Data Engineer Bootcamp (2026)

Assigned scope: 10:16:15 — Functions: Date, Text & NULL (focus NULL/COALESCE)

How to use it: Use the chapter like a paid-course class; then close it and complete the course NULL/COALESCE SQL task.

[OPEN VIDEO / SOURCE - CLICK HERE \(youtube.com\)](https://www.youtube.com/watch?v=ol9_NnC9-cc)

URL: https://www.youtube.com/watch?v=ol9_NnC9-cc

Do this after watching

Close the external source. Return to the bundled lesson and perform the exact Follow with me once steps from your own keyboard. Save evidence from your course data/tool, not a screenshot of the video.

7. Follow with me once

```
SELECT COUNT(*) AS rows_all, COUNT(sales_rep) AS rows_with_rep FROM sales_orders;
```

```
SELECT order_id, COALESCE(sales_rep,
```

```
Unassigned'
```

```
) AS rep_display FROM sales_orders WHERE sales_rep IS NULL;
```

Compare COUNT(*) with COUNT(sales_rep), find the NULL row, and display Unassigned with COALESCE without updating the table.

8. Expected result / what you should see

30 total rows; 29 non-NULL sales_rep; O1030 is the one NULL sales_rep row.

9. Why it worked

This works because find missing values correctly, explain COUNT(*) versus COUNT(column), and use COALESCE only when a display/default rule is justified is made explicit before the result is accepted. Your independent check is: IS NOT NULL count = 29. The proof is the result grain and control query, not a plausible-looking result grid.

10. Common beginner mistakes

If = NULL returns nothing, replace it with IS NULL. Do not use COALESCE to pretend missing source data is known; here it is a presentation label only.

11. If you get stuck - recovery ladder

1. Return to the lesson prerequisite/starting state; do not stack fixes on an unknown state. 2. Rebuild the mental picture: NULL is an empty "unknown/not supplied" slot, not zero or an empty string. 3. Run the earliest independent control: IS NOT NULL count = 29. 4. Repair only the first failed step, rerun that same control, and save the failed attempt before opening protected Review.

12. Now you do a small version alone

Return all rows with sales_rep IS NOT NULL and prove their count. Then show every order with a rep_label that uses Unassigned only for display.

13. Validate your result

IS NOT NULL count = 29. Source sales_rep for O1030 remains NULL.

14. Teach it back in your own words

Without notes, explain NULL, IS NULL and COALESCE in your own words. Start with the purpose: Find missing values correctly, explain COUNT(*) versus COUNT(column), and use COALESCE only when a display/default rule is justified. Then name one realistic failure mode and explain how you would prove the result using this check: IS NOT NULL count = 29.

15. Protected review + changed fresh retry

Save the genuine independent attempt first. If it is weak, open only the matching protected Review, identify the exact gap, record the repair, then close Review. Do a changed retry for NULL, IS NULL and COALESCE with a different input/identifier/condition and validate it independently. Work completed while Review is open is repair evidence, not mastery evidence.

16. Evidence / mastery + exact next route

Mastery evidence: Complete the matching guided/independent work with review material closed; save the actual result/error and validation evidence. Exact next route: DE02-07.

DE02-07 - Arithmetic expressions and safe calculations

1. Why it matters

Build auditable gross/net calculations from source columns and validate them manually. This matters because SQL can execute without error while filtering, grouping or calculating the wrong business population.

2. What you are learning

Build auditable gross/net calculations from source columns and validate them manually.

3. Prerequisite / what you already know

Prerequisite: DE02-06 mastered with saved evidence.

4. Picture it in your head

A SQL expression is a calculator applied to each output row. Safe calculation means the numerator/denominator, NULL behavior, units, and rounding rule are explicit.

5. New words before we use them

A calculated result should show its business formula. $\text{gross_sales} = \text{qty} * \text{unit_price}$. $\text{net_sales} = \text{gross_sales} * (1 - \text{discount_pct})$.

6. Useful video / source

Source: Luke Barousse — Data Engineer Bootcamp (2026)

Assigned scope: 1:39:28 — Operators & Functions

How to use it: Watch the operators/functions chapter, type examples, then solve the course safe-calculation task with the video closed.

[OPEN VIDEO / SOURCE - CLICK HERE \(youtube.com\)](https://www.youtube.com/watch?v=ol9_NnC9-cc)

URL: https://www.youtube.com/watch?v=ol9_NnC9-cc

Do this after watching

Close the external source. Return to the bundled lesson and perform the exact Follow with me once steps from your own keyboard. Save evidence from your course data/tool, not a screenshot of the video.

7. Follow with me once

```
SELECT order_id, qty, unit_price, discount_pct, qty*unit_price AS gross_sales, qty*unit_price*(1-discount_pct) AS net_sales FROM sales_orders;
```

Calculate gross_sales and net_sales for all orders. Manually verify O1002.

8. Expected result / what you should see

O1002 gross = 1350; net = 1282.50.

9. Why it worked

This works because build auditable gross/net calculations from source columns and validate them manually is made explicit before the result is accepted. Your independent check is: For every row with $\text{discount_pct} > 0$, $\text{discount_value} > 0$ and $\text{net_sales} \leq \text{gross_sales}$. The proof is the result grain and control query, not a plausible-looking result grid.

10. Common beginner mistakes

If net is larger than gross for a positive discount, inspect the sign. If decimals look strange, round for presentation only after the underlying calculation is correct.

11. If you get stuck - recovery ladder

1. Return to the lesson prerequisite/starting state; do not stack fixes on an unknown state. 2. Rebuild the mental picture: A SQL expression is a calculator applied to each output row. 3. Run the earliest independent control: For every row with `discount_pct > 0`, `discount_value > 0` and `net_sales <= gross_sales`. 4. Repair only the first failed step, rerun that same control, and save the failed attempt before opening protected Review.

12. Now you do a small version alone

Calculate `discount_value = gross_sales - net_sales` and validate two orders manually.

13. Validate your result

For every row with `discount_pct > 0`, `discount_value > 0` and `net_sales <= gross_sales`. Save two manual checks.

14. Teach it back in your own words

Without notes, explain Arithmetic expressions and safe calculations in your own words. Start with the purpose: Build auditable gross/net calculations from source columns and validate them manually. Then name one realistic failure mode and explain how you would prove the result using this check: For every row with `discount_pct > 0`, `discount_value > 0` and `net_sales <= gross_sales`.

15. Protected review + changed fresh retry

Save the genuine independent attempt first. If it is weak, open only the matching protected Review, identify the exact gap, record the repair, then close Review. Do a changed retry for Arithmetic expressions and safe calculations with a different input/identifier/condition and validate it independently. Work completed while Review is open is repair evidence, not mastery evidence.

16. Evidence / mastery + exact next route

Mastery evidence: Complete the matching guided/independent work with review material closed; save the actual result/error and validation evidence. Exact next route: DE02-08.

DE02-08 - Aggregate functions

1. Why it matters

Summarize eligible rows with COUNT, SUM, AVG, MIN and MAX and explain what NULL does to common aggregates. This matters because SQL can execute without error while filtering, grouping or calculating the wrong business population.

2. What you are learning

Summarize eligible rows with COUNT, SUM, AVG, MIN and MAX and explain what NULL does to common aggregates.

3. Prerequisite / what you already know

Prerequisite: DE02-07 mastered with saved evidence.

4. Picture it in your head

A formula is a recipe attached to a cell: it takes referenced ingredients, performs an operation, and recalculates when the ingredients change.

5. New words before we use them

Without GROUP BY, an aggregate summarizes the filtered input into one result row. COUNT(*) counts rows, while COUNT(column) ignores NULL values. SUM/AVG/MIN/MAX operate on the values that survive filtering.

6. Useful video / source

Source: Alex The Analyst — SQL Beginner to Advanced

Assigned scope: 33:54–44:18

How to use it: Watch the assigned visual first, then use the Teaching Book for the course-specific build. Close the visual before the independent task.

[OPEN VIDEO / SOURCE - CLICK HERE \(youtube.com\)](https://www.youtube.com/watch?v=OT1RErkfLNQ)

URL: <https://www.youtube.com/watch?v=OT1RErkfLNQ>

Do this after watching

Close the external source. Return to the bundled lesson and perform the exact Follow with me once steps from your own keyboard. Save evidence from your course data/tool, not a screenshot of the video.

7. Follow with me once

```
SELECT COUNT(*) AS order_rows, SUM(qty) AS total_qty, AVG(unit_price) AS avg_unit_price, MIN(unit_price) AS min_price, MAX(unit_price) AS max_price, SUM(qty*unit_price) AS gross_sales FROM sales_orders;
```

Return row count, total qty, average price, min/max price and gross sales for all rows.

8. Expected result / what you should see

30 rows; total qty 105; average unit_price about 666.33; gross sales 50000.

9. Why it worked

The calculation is trustworthy because the output grain is declared before aggregation and the grouped result is reconciled to an independent detail-level control. Your independent check is: Completed order count = 27; total qty = 100; total net_sales = 42984.50. The proof is the result grain and control query, not a plausible-looking result grid.

10. Common beginner mistakes

If an aggregate is unexpectedly high/low, first inspect the eligible row count and whether the filter matches the business rule.

11. If you get stuck - recovery ladder

1. Return to the lesson prerequisite/starting state; do not stack fixes on an unknown state. 2. Rebuild the mental picture: A formula is a recipe attached to a cell: it takes referenced ingredients, performs an operation, and recalculates when the ingredients change. 3. Run the earliest independent control: Completed order count = 27; total qty = 100; total net_sales = 42984.50. 4. Repair only the first failed step, rerun that same control, and save the failed attempt before opening protected Review.

12. Now you do a small version alone

For Completed orders only, return order count, total qty and total net_sales in one summary row.

13. Validate your result

Completed order count = 27; total qty = 100; total net_sales = 42984.50.

14. Teach it back in your own words

Without notes, explain Aggregate functions in your own words. Start with the purpose: Summarize eligible rows with COUNT, SUM, AVG, MIN and MAX and explain what NULL does to common aggregates. Then name one realistic failure mode and explain how you would prove the result using this check: Completed order count = 27; total qty = 100; total net_sales = 42984.50.

15. Protected review + changed fresh retry

Save the genuine independent attempt first. If it is weak, open only the matching protected Review, identify the exact gap, record the repair, then close Review. Do a changed retry for Aggregate functions with a different input/identifier/condition and validate it independently. Work completed while Review is open is repair evidence, not mastery evidence.

16. Evidence / mastery + exact next route

Mastery evidence: Complete the matching guided/independent work with review material closed; save the actual result/error and validation evidence. Exact next route: DE02-09.

DE02-09 - GROUP BY and grouped grain

1. Why it matters

Create one summary row per group, state output grain first, and reconcile grouped totals to an overall control. This matters because SQL can execute without error while filtering, grouping or calculating the wrong business population.

2. What you are learning

Create one summary row per group, state output grain first, and reconcile grouped totals to an overall control.

3. Prerequisite / what you already know

Prerequisite: DE02-08 mastered with saved evidence.

4. Picture it in your head

Think of grain as camera zoom: "one row per order" and "one row per order item" are different zoom levels. A calculation is trustworthy only when you know which level the row represents.

5. New words before we use them

GROUP BY changes result grain. If you GROUP BY region, one output row represents one region. Selected non-aggregate columns must belong to the grouping grain.

6. Useful video / source

Source: Alex The Analyst — SQL Beginner to Advanced

Assigned scope: 33:54–44:18

How to use it: Watch the assigned visual first, then use the Teaching Book for the course-specific build. Close the visual before the independent task.

[OPEN VIDEO / SOURCE - CLICK HERE \(youtube.com\)](https://www.youtube.com/watch?v=OT1RErkfLNQ)

URL: <https://www.youtube.com/watch?v=OT1RErkfLNQ>

Do this after watching

Close the external source. Return to the bundled lesson and perform the exact Follow with me once steps from your own keyboard. Save evidence from your course data/tool, not a screenshot of the video.

7. Follow with me once

```
SELECT region, COUNT(*) AS orders, SUM(qty) AS total_qty, SUM(qty*unit_price) AS gross_sales FROM sales_orders
WHERE status='
```

Completed'

GROUP BY region ORDER BY region;

Build Completed order count and gross sales by region.

8. Expected result / what you should see

East 8 / 12800; North 5 / 9310; South 7 / 12010; West 7 / 10930.

9. Why it worked

The calculation is trustworthy because the output grain is declared before aggregation and the grouped result is reconciled to an independent detail-level control. Your independent check is: 3 result rows; grouped net total reconciles to 42984.50. The proof is the result grain and control query, not a plausible-looking result grid.

10. Common beginner mistakes

If grouped totals do not reconcile, run the same measure with the same WHERE filter without GROUP BY. Do not trust plausible-looking group rows.

11. If you get stuck - recovery ladder

1. Return to the lesson prerequisite/starting state; do not stack fixes on an unknown state. 2. Rebuild the mental picture: Think of grain as camera zoom: “one row per order” and “one row per order item” are different zoom levels. 3. Run the earliest independent control: 3 result rows; grouped net total reconciles to 42984.50. 4. Repair only the first failed step, rerun that same control, and save the failed attempt before opening protected Review.

12. Now you do a small version alone

Build Completed order count, qty and net_sales by payment_method and state the result grain.

13. Validate your result

3 result rows; grouped net total reconciles to 42984.50.

14. Teach it back in your own words

Without notes, explain GROUP BY and grouped grain in your own words. Start with the purpose: Create one summary row per group, state output grain first, and reconcile grouped totals to an overall control. Then name one realistic failure mode and explain how you would prove the result using this check: 3 result rows; grouped net total reconciles to 42984.50.

15. Protected review + changed fresh retry

Save the genuine independent attempt first. If it is weak, open only the matching protected Review, identify the exact gap, record the repair, then close Review. Do a changed retry for GROUP BY and grouped grain with a different input/identifier/condition and validate it independently. Work completed while Review is open is repair evidence, not mastery evidence.

16. Evidence / mastery + exact next route

Mastery evidence: Complete the matching guided/independent work with review material closed; save the actual result/error and validation evidence. Exact next route: DE02-10.

DE02-10 - HAVING after aggregation

1. Why it matters

Use WHERE for source-row eligibility and HAVING for conditions on grouped/aggregate results. This matters because SQL can execute without error while filtering, grouping or calculating the wrong business population.

2. What you are learning

Use WHERE for source-row eligibility and HAVING for conditions on grouped/aggregate results.

3. Prerequisite / what you already know

Prerequisite: DE02-09 mastered with saved evidence.

4. Picture it in your head

WHERE filters rows before grouping; HAVING filters the summary buckets after aggregation. Picture two gates on opposite sides of the GROUP BY step.

5. New words before we use them

WHERE runs before aggregation and filters source rows. HAVING filters groups after GROUP BY. A condition such as status=Completed belongs in WHERE; a condition such as SUM(qty)>=25 belongs in HAVING.

6. Useful video / source

Source: Alex The Analyst — SQL Beginner to Advanced

Assigned scope: 44:18–47:41

How to use it: Watch the assigned visual first, then use the Teaching Book for the course-specific build. Close the visual before the independent task.

[OPEN VIDEO / SOURCE - CLICK HERE \(youtube.com\)](https://www.youtube.com/watch?v=OT1RErkfLNQ)

URL: <https://www.youtube.com/watch?v=OT1RErkfLNQ>

Do this after watching

Close the external source. Return to the bundled lesson and perform the exact Follow with me once steps from your own keyboard. Save evidence from your course data/tool, not a screenshot of the video.

7. Follow with me once

```
SELECT region, SUM(qty) AS total_qty FROM sales_orders GROUP BY region HAVING SUM(qty) >
= 25 ORDER BY total_qty DESC;
```

Group all rows by region and keep groups with SUM(qty) >= 25.

8. Expected result / what you should see

East 29, North 27, West 25; South is excluded.

9. Why it worked

The calculation is trustworthy because the output grain is declared before aggregation and the grouped result is reconciled to an independent detail-level control. Your independent check is: Cash 39 and UPI 39 qualify; Card 22 does not. The proof is the result grain and control query, not a plausible-looking result grid.

10. Common beginner mistakes

If you put SUM(qty) in WHERE, move the aggregate condition to HAVING. If business eligibility is wrong, inspect WHERE separately.

11. If you get stuck - recovery ladder

1. Return to the lesson prerequisite/starting state; do not stack fixes on an unknown state. 2. Rebuild the mental picture: WHERE filters rows before grouping; HAVING filters the summary buckets after aggregation. 3. Run the earliest independent control: Cash 39 and UPI 39 qualify; Card 22 does not. 4. Repair only the first failed step, rerun that same control, and save the failed attempt before opening protected Review.

12. Now you do a small version alone

For Completed orders only, return payment methods whose total qty is at least 25.

13. Validate your result

Cash 39 and UPI 39 qualify; Card 22 does not.

14. Teach it back in your own words

Without notes, explain HAVING after aggregation in your own words. Start with the purpose: Use WHERE for source-row eligibility and HAVING for conditions on grouped/aggregate results. Then name one realistic failure mode and explain how you would prove the result using this check: Cash 39 and UPI 39 qualify; Card 22 does not.

15. Protected review + changed fresh retry

Save the genuine independent attempt first. If it is weak, open only the matching protected Review, identify the exact gap, record the repair, then close Review. Do a changed retry for HAVING after aggregation with a different input/identifier/condition and validate it independently. Work completed while Review is open is repair evidence, not mastery evidence.

16. Evidence / mastery + exact next route

Mastery evidence: Complete the matching guided/independent work with review material closed; save the actual result/error and validation evidence. Exact next route: DE02-11.

DE02-11 - CASE for business categories

1. Why it matters

Translate business rules into ordered CASE branches and validate bucket coverage. This matters because SQL can execute without error while filtering, grouping or calculating the wrong business population.

2. What you are learning

Translate business rules into ordered CASE branches and validate bucket coverage.

3. Prerequisite / what you already know

Prerequisite: DE02-10 mastered with saved evidence.

4. Picture it in your head

CASE is a rule-based label maker: each row is tested from top to bottom and receives the first matching category, so rule order and coverage matter.

5. New words before we use them

CASE tests WHEN branches from top to bottom and returns the first matching result. For overlapping thresholds, put the highest/more restrictive rule first.

6. Useful video / source

Source: Alex The Analyst — SQL Beginner to Advanced

Assigned scope: 1:26:39–1:35:16

How to use it: Watch the assigned visual first, then use the Teaching Book for the course-specific build. Close the visual before the independent task.

[OPEN VIDEO / SOURCE - CLICK HERE \(youtube.com\)](https://www.youtube.com/watch?v=OT1RErkfLNQ)

URL: <https://www.youtube.com/watch?v=OT1RErkfLNQ>

Do this after watching

Close the external source. Return to the bundled lesson and perform the exact Follow with me once steps from your own keyboard. Save evidence from your course data/tool, not a screenshot of the video.

7. Follow with me once

```
SELECT order_id, qty*unit_price AS gross_sales, CASE WHEN qty*unit_price >
= 2500 THEN '
High'
WHEN qty*unit_price >
= 1500 THEN '
Medium'
ELSE '
Small'
END AS order_size FROM sales_orders;
Bucket all orders into High >
=2500, Medium >
=1500, otherwise Small, then count each bucket.
```

8. Expected result / what you should see

High 3; Medium 14; Small 13.

9. Why it worked

The query is trustworthy because the business question, row grain and transformation stages are explicit, then an independent row/key/total check challenges a plausible-but-wrong result. Your independent check is: Counts across the three groups sum to 30 and source status values remain unchanged. The proof is the result grain and control query, not a plausible-looking result grid.

10. Common beginner mistakes

If High is empty, inspect branch order. If counts do not sum to 30, inspect missing ELSE or filter logic.

11. If you get stuck - recovery ladder

1. Return to the lesson prerequisite/starting state; do not stack fixes on an unknown state. 2. Rebuild the mental picture: CASE is a rule-based label maker: each row is tested from top to bottom and receives the first matching category, so rule order and coverage matter. 3. Run the earliest independent control: Counts across the three groups sum to 30 and source status values remain unchanged. 4. Repair only the first failed step, rerun that same control, and save the failed attempt before opening protected Review.

12. Now you do a small version alone

Create fulfillment_group: Completed -> Closed; Pending -> Open; otherwise Other. Count each group.

13. Validate your result

Counts across the three groups sum to 30 and source status values remain unchanged.

14. Teach it back in your own words

Without notes, explain CASE for business categories in your own words. Start with the purpose: Translate business rules into ordered CASE branches and validate bucket coverage. Then name one realistic failure mode and explain how you would prove the result using this check: Counts across the three groups sum to 30 and source status values remain unchanged.

15. Protected review + changed fresh retry

Save the genuine independent attempt first. If it is weak, open only the matching protected Review, identify the exact gap, record the repair, then close Review. Do a changed retry for CASE for business categories with a different input/identifier/condition and validate it independently. Work completed while Review is open is repair evidence, not mastery evidence.

16. Evidence / mastery + exact next route

Mastery evidence: Complete the matching guided/independent work with review material closed; save the actual result/error and validation evidence. Exact next route: DE02-12.

DE02-12 - String and date basics

1. Why it matters

Use basic text cleanup/display functions and extract date parts while preserving source data. This matters because SQL can execute without error while filtering, grouping or calculating the wrong business population.

2. What you are learning

Use basic text cleanup/display functions and extract date parts while preserving source data.

3. Prerequisite / what you already know

Prerequisite: DE02-11 mastered with saved evidence.

4. Picture it in your head

String functions reshape text labels; date functions place records on a calendar/time axis. Both are safest when the original value and transformed value can be compared.

5. New words before we use them

TRIM removes surrounding whitespace from a result expression, UPPER changes display casing, and YEAR/MONTH extract calendar components in MySQL. These query expressions do not update source values.

6. Useful video / source

Source: Alex The Analyst — SQL Beginner to Advanced

Assigned scope: 1:15:10–1:26:39

How to use it: Watch the assigned visual first, then use the Teaching Book for the course-specific build. Close the visual before the independent task.

[OPEN VIDEO / SOURCE - CLICK HERE \(youtube.com\)](https://www.youtube.com/watch?v=OT1RErkfLNQ)

URL: <https://www.youtube.com/watch?v=OT1RErkfLNQ>

Do this after watching

Close the external source. Return to the bundled lesson and perform the exact Follow with me once steps from your own keyboard. Save evidence from your course data/tool, not a screenshot of the video.

7. Follow with me once

```
SELECT order_id, UPPER(TRIM(region)) AS region_clean, YEAR(order_date) AS order_year, MONTH(order_date) AS order_month FROM sales_orders WHERE order_date BETWEEN '2026-07-10'
```

```
AND '
```

```
2026-07-12'
```

```
;
```

Run the date-range/text query for July 10 through July 12 inclusive.

8. Expected result / what you should see

6 rows; every order_year = 2026 and order_month = 7.

9. Why it worked

This works because use basic text cleanup/display functions and extract date parts while preserving source data is made explicit before the result is accepted. Your independent check is: Only July 13, 14 and 15 rows appear; transformed output is traceable to original text. The proof is the result grain and control query, not a plausible-looking result grid.

10. Common beginner mistakes

If dates are missing, check endpoints and data type. If text cleanup changes the business meaning, do not treat it as a safe standardization without a rule.

11. If you get stuck - recovery ladder

1. Return to the lesson prerequisite/starting state; do not stack fixes on an unknown state. 2. Rebuild the mental picture: String functions reshape text labels; date functions place records on a calendar/time axis. 3. Run the earliest independent control: Only July 13, 14 and 15 rows appear; transformed output is traceable to original text. 4. Repair only the first failed step, rerun that same control, and save the failed attempt before opening protected Review.

12. Now you do a small version alone

Return product labels as UPPER(product) and extract day-of-month for orders on July 13-15. Keep original product beside the transformed value.

13. Validate your result

Only July 13, 14 and 15 rows appear; transformed output is traceable to original text.

14. Teach it back in your own words

Without notes, explain String and date basics in your own words. Start with the purpose: Use basic text cleanup/display functions and extract date parts while preserving source data. Then name one realistic failure mode and explain how you would prove the result using this check: Only July 13, 14 and 15 rows appear; transformed output is traceable to original text.

15. Protected review + changed fresh retry

Save the genuine independent attempt first. If it is weak, open only the matching protected Review, identify the exact gap, record the repair, then close Review. Do a changed retry for String and date basics with a different input/identifier/condition and validate it independently. Work completed while Review is open is repair evidence, not mastery evidence.

16. Evidence / mastery + exact next route

Mastery evidence: Complete the matching guided/independent work with review material closed; save the actual result/error and validation evidence. Exact next route: DE02-13.

DE02-13 - Simple subqueries and IN/EXISTS awareness

1. Why it matters

Use a simple scalar subquery and recognize IN/EXISTS as ways to filter based on another query result. This matters because SQL can execute without error while filtering, grouping or calculating the wrong business population.

2. What you are learning

Use a simple scalar subquery and recognize IN/EXISTS as ways to filter based on another query result.

3. Prerequisite / what you already know

Prerequisite: DE02-12 mastered with saved evidence.

4. Picture it in your head

A subquery is a question nested inside a larger question. The inner result becomes an input condition for the outer query, so you should be able to run and validate it separately.

5. New words before we use them

A subquery is a query nested inside another query. A scalar subquery returns one value, such as an average. IN compares a value against a set returned by a subquery.

6. Useful video / source

Source: Alex The Analyst — SQL Beginner to Advanced

Assigned scope: 1:35:16–1:45:58

How to use it: Watch the assigned visual first, then use the Teaching Book for the course-specific build. Close the visual before the independent task.

[OPEN VIDEO / SOURCE - CLICK HERE \(youtube.com\)](https://www.youtube.com/watch?v=OT1RErkfLNQ)

URL: <https://www.youtube.com/watch?v=OT1RErkfLNQ>

Do this after watching

Close the external source. Return to the bundled lesson and perform the exact Follow with me once steps from your own keyboard. Save evidence from your course data/tool, not a screenshot of the video.

7. Follow with me once

```
SELECT order_id, qty*unit_price*(1-discount_pct) AS net_sales FROM sales_orders WHERE status='Completed'
```

```
AND qty*unit_price*(1-discount_pct) >
```

```
( SELECT AVG(qty*unit_price*(1-discount_pct)) FROM sales_orders WHERE status='Completed'
```

```
) ORDER BY net_sales DESC;
```

Find Completed orders whose net_sales is above the average Completed net_sales. Run the inner average query alone first.

8. Expected result / what you should see

Average Completed net_sales is about 1592.02; 13 orders are above it.

9. Why it worked

This works because use a simple scalar subquery and recognize IN/EXISTS as ways to filter based on another query result is made explicit before the result is accepted. Your independent check is: The inner set is East, North, West; no South order appears in the outer result. The proof is the result grain and control query, not a plausible-looking result grid.

10. Common beginner mistakes

If the outer result is wrong, run the inner query independently and verify it returns one value. Keep the same Completed eligibility in both places.

11. If you get stuck - recovery ladder

1. Return to the lesson prerequisite/starting state; do not stack fixes on an unknown state. 2. Rebuild the mental picture: A subquery is a question nested inside a larger question. 3. Run the earliest independent control: The inner set is East, North, West; no South order appears in the outer result. 4. Repair only the first failed step, rerun that same control, and save the failed attempt before opening protected Review.

12. Now you do a small version alone

Use IN with a subquery to return orders whose region belongs to a region whose all-status SUM(qty) is at least 25. State what the inner query returns.

13. Validate your result

The inner set is East, North, West; no South order appears in the outer result.

14. Teach it back in your own words

Without notes, explain Simple subqueries and IN/EXISTS awareness in your own words. Start with the purpose: Use a simple scalar subquery and recognize IN/EXISTS as ways to filter based on another query result. Then name one realistic failure mode and explain how you would prove the result using this check: The inner set is East, North, West; no South order appears in the outer result.

15. Protected review + changed fresh retry

Save the genuine independent attempt first. If it is weak, open only the matching protected Review, identify the exact gap, record the repair, then close Review. Do a changed retry for Simple subqueries and IN/EXISTS awareness with a different input/identifier/condition and validate it independently. Work completed while Review is open is repair evidence, not mastery evidence.

16. Evidence / mastery + exact next route

Mastery evidence: Complete the matching guided/independent work with review material closed; save the actual result/error and validation evidence. Exact next route: DE02-14.

DE02-14 - Debugging and validation workflow

1. Why it matters

Debug syntax/logic problems systematically and prove a result with independent counts, grain checks and reconciliations rather than trusting a green Run button. This matters because SQL can execute without error while filtering, grouping or calculating the wrong business population.

2. What you are learning

Debug syntax/logic problems systematically and prove a result with independent counts, grain checks and reconciliations rather than trusting a green Run button.

3. Prerequisite / what you already know

Prerequisite: DE02-13 mastered with saved evidence.

4. Picture it in your head

Debug SQL like a pipeline of checkpoints: prove the source rows, then filters, joins, grouping, and final totals one stage at a time instead of staring at the final query.

5. New words before we use them

Debugging has two branches: the query fails to run, or it runs but the answer is wrong. For syntax errors, read the exact error and reduce the query. For wrong results, verify source grain, filters, row counts and aggregates.

6. Useful video / source

Bundled visual / source

Course research bundle — SQL troubleshooting + wrong-result validation | Microsoft troubleshooting process video + Metabase duplicate/wrong-result logic guide; then course-owned validation lab Course path: supplements/SQL_DEBUGGING_VALIDATION_VIDEO_BUNDLE_RC1.html Use the short troubleshooting video for the systematic debugging mindset, then the duplicate-result guide for logic validation. Close sources and debug the course query independently.

7. Follow with me once

```
-- Wrong business rule: Pending rows are accidentally included
SELECT region, SUM(qty*unit_price) AS gross_sales
FROM sales_orders WHERE status <
>
```

Cancelled'

GROUP BY region;

```
-- Control SELECT COUNT(*) FROM sales_orders WHERE status='
```

Completed'

;

Run the wrong-business-rule query, compare its eligibility count with Completed-only count, repair the filter, then reconcile grouped gross_sales to a separate Completed overall total.

8. Expected result / what you should see

Completed-only count = 27. Completed grouped gross total reconciles to 45050. A query that includes Pending is not equivalent even if the numbers look plausible.

9. Why it worked

This works because debug syntax/logic problems systematically and prove a result with independent counts, grain checks and reconciliations rather than trusting a green Run button is made explicit before the result is accepted. Your independent check is: The corrected query has the intended status and regions only. The proof is the result grain and control query, not a plausible-looking result grid.

10. Common beginner mistakes

Do not randomly edit SQL. Record expected result, actual result, exact error/wrong signal, one hypothesis, one change and rerun evidence. Preserve the failed attempt.

11. If you get stuck - recovery ladder

1. Return to the lesson prerequisite/starting state; do not stack fixes on an unknown state. 2. Rebuild the mental picture: Debug SQL like a pipeline of checkpoints: prove the source rows, then filters, joins, grouping, and final totals one stage at a time instead of staring at the final query. 3. Run the earliest independent control: The corrected query has the intended status and regions only. 4. Repair only the first failed step, rerun that same control, and save the failed attempt before opening protected Review.

12. Now you do a small version alone

Take a deliberately wrong query that uses OR without parentheses, diagnose why a Pending row leaks into the output, repair it, and save before/after row counts plus the corrected English rule.

13. Validate your result

The corrected query has the intended status and regions only. A separate COUNT/control query supports the result.

14. Teach it back in your own words

Without notes, explain Debugging and validation workflow in your own words. Start with the purpose: Debug syntax/logic problems systematically and prove a result with independent counts, grain checks and reconciliations rather than trusting a green Run button. Then name one realistic failure mode and explain how you would prove the result using this check: The corrected query has the intended status and regions only.

15. Protected review + changed fresh retry

Save the genuine independent attempt first. If it is weak, open only the matching protected Review, identify the exact gap, record the repair, then close Review. Do a changed retry for Debugging and validation workflow with a different input/identifier/condition and validate it independently. Work completed while Review is open is repair evidence, not mastery evidence.

16. Evidence / mastery + exact next route

Mastery evidence: Complete the matching guided/independent work with review material closed; save the actual result/error and validation evidence. Exact next route: module practical / Gate route after the mastery checklist.